Part 1: Tuple Programming Questions

Tuples are ordered and immutable. Unless a question says otherwise, display results in the format shown.

1. Create a tuple containing 10 numbers and print all elements using a loop.

SAMPLE INPUT
```
numbers = (2, 4, 6, 8, 10, 12, 14, 16, 18, 20)
```

EXPECTED OUTPUT
```
2
4
6
8
10
12
14
16
18
20
```

2. Create a tuple of student names and print the first, last, and middle elements.

SAMPLE INPUT
```
students = ('Asha', 'Rahul', 'Meena', 'Kiran', 'Ravi')
```

EXPECTED OUTPUT
```
First: Asha
Last: Ravi
Middle: Meena
```

3. Find the length of a tuple without using len().

SAMPLE INPUT
```
numbers = (5, 10, 15, 20, 25, 30)
```

EXPECTED OUTPUT
```
Length: 6
```

4. Count how many times a given number appears in a tuple without using count().

SAMPLE INPUT
```
numbers = (10, 20, 10, 30, 10, 40)
target = 10
```

EXPECTED OUTPUT
```
Occurrences of 10: 3
```

5. Find the index of a given element without using index(). If it is not found, display a suitable message.

SAMPLE INPUT
```
numbers = (12, 24, 36, 48, 60)
target = 36
```

EXPECTED OUTPUT
```
Index: 2
```

6. Create a tuple containing different data types and print the type of each element.

SAMPLE INPUT
```
values = ('Python', 25, 8.5, True)
```

EXPECTED OUTPUT
```
Python: str
25: int
8.5: float
True: bool
```

7. Reverse a tuple without using a loop.

SAMPLE INPUT
```
numbers = (10, 20, 30, 40, 50)
```

EXPECTED OUTPUT
```
(50, 40, 30, 20, 10)
```

8. Extract the first five elements and the last three elements from a tuple.

SAMPLE INPUT
```
numbers = (10, 20, 30, 40, 50, 60, 70, 80)
```

EXPECTED OUTPUT
```
First five: (10, 20, 30, 40, 50)
Last three: (60, 70, 80)
```

9. Find the largest and smallest numbers in a tuple without using max() and min().

SAMPLE INPUT
```
numbers = (45, 12, 78, 3, 56, 21)
```

EXPECTED OUTPUT
```
Largest: 78
Smallest: 3
```

10. Convert a list into a tuple and a tuple into a list.

SAMPLE INPUT
```
items_list = ['pen', 'book', 'bag']
items_tuple = ('red', 'green', 'blue')
```

EXPECTED OUTPUT
```
Tuple: ('pen', 'book', 'bag')
List: ['red', 'green', 'blue']
```

11. Find the sum of all elements in a tuple without using sum().

SAMPLE INPUT
```
numbers = (10, 20, 30, 40, 50)
```

EXPECTED OUTPUT
```
Sum: 150
```

12. Find the second-largest unique number in a tuple.

SAMPLE INPUT
```
numbers = (12, 45, 32, 45, 28, 32, 40)
```

EXPECTED OUTPUT
```
Second-largest unique number: 40
```

13. Remove duplicate elements from a tuple while preserving their original order.

SAMPLE INPUT
```
numbers = (4, 2, 4, 7, 2, 9, 7, 1)
```

EXPECTED OUTPUT
```
(4, 2, 7, 9, 1)
```

14. Create two tuples and concatenate them without using the + operator.

SAMPLE INPUT
```
first = (1, 2, 3)
second = (4, 5, 6)
```

EXPECTED OUTPUT
```
(1, 2, 3, 4, 5, 6)
```

15. Find all positions where a particular number appears in a tuple.

SAMPLE INPUT
```
numbers = (5, 8, 5, 2, 5, 9)
target = 5
```

EXPECTED OUTPUT

```
Positions: [0, 2, 4]
```

16. Separate even and odd numbers from a tuple into two new tuples.

SAMPLE INPUT
```
numbers = (3, 8, 11, 14, 17, 20, 23, 26)
```

EXPECTED OUTPUT
```
Even: (8, 14, 20, 26)
Odd: (3, 11, 17, 23)
```

17. Check whether a tuple is a palindrome.

SAMPLE INPUT
```
values = (1, 2, 3, 2, 1)
```

EXPECTED OUTPUT
```
Palindrome: True
```

18. Find the common elements between two tuples without converting them into sets.

SAMPLE INPUT
```
first = (10, 20, 30, 40)
second = (30, 40, 50, 60)
```

EXPECTED OUTPUT
```
Common elements: (30, 40)
```

19. Sort a tuple of numbers in ascending order without using sorted().

SAMPLE INPUT
```
numbers = (35, 12, 48, 7, 23)
```

EXPECTED OUTPUT
```
(7, 12, 23, 35, 48)
```

20. Find the elements that appear exactly once in a tuple.

SAMPLE INPUT
```
numbers = (2, 3, 2, 5, 7, 3, 9, 10, 9)
```

EXPECTED OUTPUT
```
Elements appearing once: (5, 7, 10)
```

21. For student marks, calculate the average, highest mark, lowest mark, and number of students who passed (marks >= 35).

SAMPLE INPUT
```
marks = (85, 32, 92, 67, 28, 75)
```

EXPECTED OUTPUT
```
Average: 63.17
Highest: 92
Lowest: 28
Passed: 4
```

22. Find all pairs in a tuple whose sum equals a target value.

SAMPLE INPUT
```
numbers = (1, 2, 3, 4, 5, 6)
target = 7
```

EXPECTED OUTPUT
```
Pairs: [(1, 6), (2, 5), (3, 4)]
```

23. Find the maximum difference between any two elements in a tuple.

SAMPLE INPUT
```
numbers = (14, 3, 27, 9, 18)
```

EXPECTED OUTPUT
```
Maximum difference: 24
```

24. Rotate a tuple to the right by k positions.

SAMPLE INPUT
```
numbers = (1, 2, 3, 4, 5)
k = 2
```

EXPECTED OUTPUT
```
(4, 5, 1, 2, 3)
```

25. Given employee records (id, name, salary), display employees earning more than ₹40,000.

SAMPLE INPUT
```
employees = ((101, 'Asha', 42000), (102, 'Rahul', 38000), (103, 'Meena', 55000))
```

EXPECTED OUTPUT
```
101 Asha 42000
103 Meena 55000
```

26. Convert a tuple of key-value pairs into a dictionary.

SAMPLE INPUT
```
pairs = (('name', 'Asha'), ('age', 21), ('course', 'Python'))
```

EXPECTED OUTPUT
```
{'name': 'Asha', 'age': 21, 'course': 'Python'}
```

27. Unpack a tuple containing a student's name, age, marks, and course into separate variables.

SAMPLE INPUT
```
student = ('Kiran', 20, 88, 'Java')
```

EXPECTED OUTPUT
```
Name: Kiran
Age: 20
Marks: 88
Course: Java
```

28. Given nested tuples of product names and prices, calculate the total price.

SAMPLE INPUT
```
products = (('Pen', 10), ('Book', 80), ('Bag', 650))
```

EXPECTED OUTPUT
```
Total price: 740
```

29. Find the longest consecutive increasing sequence in a tuple. Return the sequence, not just its length.

SAMPLE INPUT
```
numbers = (1, 2, 5, 6, 7, 3, 4, 8)
```

EXPECTED OUTPUT
```
Longest increasing sequence: (5, 6, 7)
Length: 3
```

30. Given daily temperatures, identify the days when temperature was above the average.

SAMPLE INPUT
```
temperatures = (30, 32, 29, 35, 31)
Day numbers start at 1.
```

EXPECTED OUTPUT
```
Average: 31.4
Days above average: Day 2 (32), Day 4 (35)
```

Part 2: Set Programming Questions

Sets contain unique elements and do not guarantee display order. The order of elements in set output may vary.

31. Create a set from a list containing duplicate numbers and display the unique values.

SAMPLE INPUT
```
numbers = [1, 2, 2, 3, 4, 4, 5]
```

EXPECTED OUTPUT
```
Unique values: {1, 2, 3, 4, 5}
```

32. Add five elements to a set using add() and update().

SAMPLE INPUT
```
Start with: values = {10, 20}
Add 30 using add(); add 40, 50, 60 using update().
```

EXPECTED OUTPUT
```
{10, 20, 30, 40, 50, 60}
```

33. Remove an element using remove() and discard(). Observe the behavior for a missing element.

SAMPLE INPUT
```
values = {10, 20, 30}
Remove 20, then discard 99.
```

EXPECTED OUTPUT
```
After remove(20): {10, 30}
After discard(99): {10, 30}
Note: remove(99) would raise KeyError.
```

34. Find the number of unique elements in a list using a set.

SAMPLE INPUT
```
numbers = [4, 4, 5, 6, 5, 7, 8, 8]
```

EXPECTED OUTPUT
```
Number of unique elements: 5
```

35. Check whether a given number exists in a set.

SAMPLE INPUT
```
numbers = {10, 20, 30, 40}
target = 30
```

EXPECTED OUTPUT
```
30 exists: True
```

36. Find the union of two sets using union() and the | operator.

SAMPLE INPUT
```
A = {1, 2, 3}
B = {3, 4, 5}
```

EXPECTED OUTPUT
```
Union: {1, 2, 3, 4, 5}
```

37. Find the intersection of two sets using intersection() and the & operator.

SAMPLE INPUT
```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

EXPECTED OUTPUT
```
Intersection: {3, 4}
```

38. Find the difference between two sets in both directions.

SAMPLE INPUT

```
A = {10, 20, 30, 40}
B = {30, 40, 50}
```

EXPECTED OUTPUT

```
A - B: {10, 20}
B - A: {50}
```

39. Find the symmetric difference between two sets.

SAMPLE INPUT

```
A = {'red', 'blue', 'green'}
B = {'blue', 'yellow', 'green'}
```

EXPECTED OUTPUT

```
Symmetric difference: {'red', 'yellow'}
```

40. Check whether one set is a subset and superset of another set.

SAMPLE INPUT

```
A = {2, 4}
B = {2, 4, 6, 8}
```

EXPECTED OUTPUT

```
A is subset of B: True
B is superset of A: True
```

41. Find all duplicate elements in a list using sets.

SAMPLE INPUT

```
numbers = [1, 2, 3, 2, 4, 1, 5, 3, 3]
```

EXPECTED OUTPUT

```
Duplicate elements: {1, 2, 3}
```

42. Find common elements between three lists.

SAMPLE INPUT

```
A = [1, 2, 3, 4, 5]
B = [2, 4, 6, 8]
C = [0, 2, 4, 9]
```

EXPECTED OUTPUT

```
Common elements: {2, 4}
```

43. Find the elements present in the first list but absent from the second list.

SAMPLE INPUT

```
first = [10, 20, 30, 40]
second = [20, 40, 50]
```

EXPECTED OUTPUT

```
Elements only in first list: {10, 30}
```

44. Given two course-registration lists, identify students registered for exactly one course.

SAMPLE INPUT

```
Python = {'Asha', 'Rahul', 'Meena'}
Java = {'Rahul', 'Meena', 'Arun'}
```

EXPECTED OUTPUT

```
Exactly one course: {'Asha', 'Arun'}
```

45. Find all unique characters in a string using a set.

SAMPLE INPUT

```
text = 'programming'
```

EXPECTED OUTPUT

```
Unique characters: {'p', 'r', 'o', 'g', 'a', 'm', 'i', 'n'}
```

46. Check whether two strings contain exactly the same unique characters.

SAMPLE INPUT
```
first = 'listen'
second = 'silent'
```

EXPECTED OUTPUT
```
Same unique characters: True
```

47. Find the missing numbers between 1 and 20 from a given list.

SAMPLE INPUT
```
numbers = [1, 2, 3, 5, 6, 9, 10, 11, 15, 20]
```

EXPECTED OUTPUT
```
Missing numbers: [4, 7, 8, 12, 13, 14, 16, 17, 18, 19]
```

48. Given two exam-participant sets, find participants who attended both exams.

SAMPLE INPUT
```
ExamA = {'Asha', 'Rahul', 'Kiran'}
ExamB = {'Rahul', 'Meena', 'Kiran'}
```

EXPECTED OUTPUT
```
Attended both: {'Rahul', 'Kiran'}
```

49. Find common words between two sentences, ignoring uppercase and lowercase differences.

SAMPLE INPUT
```
sentence1 = 'Python is Easy to Learn'
sentence2 = 'I learn PYTHON and Java'
```

EXPECTED OUTPUT
```
Common words: {'python', 'learn'}
```

50. Remove duplicate words from a sentence while preserving original word order.

SAMPLE INPUT
```
sentence = 'python is easy and python is useful'
```

EXPECTED OUTPUT
```
python is easy and useful
```

51. Find students enrolled in all three courses.

SAMPLE INPUT
```
Python = {'Asha', 'Rahul', 'Kiran'}
Java = {'Rahul', 'Kiran', 'Meena'}
React = {'Rahul', 'Kiran', 'Ravi'}
```

EXPECTED OUTPUT
```
Students in all three: {'Rahul', 'Kiran'}
```

52. Given two event registration lists, find students registered for both events and only one event.

SAMPLE INPUT
```
EventA = {'Asha', 'Rahul', 'Kiran'}
EventB = {'Rahul', 'Meena', 'Kiran'}
```

EXPECTED OUTPUT
```
Both events: {'Rahul', 'Kiran'}
Only one event: {'Asha', 'Meena'}
```

53. Given purchased and available product IDs, identify unavailable products.

SAMPLE INPUT
```
purchased = {101, 102, 105, 110}
available = {101, 102, 103, 104, 105}
```

EXPECTED OUTPUT
```
Unavailable products: {110}
```

54. Given the subjects selected by two students, find subjects selected by one student but not the other.

SAMPLE INPUT

```
Asha = {'Maths', 'Physics', 'Chemistry'}
Rahul = {'Maths', 'Biology', 'Chemistry'}
```

EXPECTED OUTPUT

```
Different subjects: {'Physics', 'Biology'}
```

55. Given visitor sets from two days, find returning visitors, new visitors, and visitors who did not return.

SAMPLE INPUT

```
Day1 = {'Asha', 'Rahul', 'Kiran'}
Day2 = {'Rahul', 'Kiran', 'Meena'}
```

EXPECTED OUTPUT

```
Returning visitors: {'Rahul', 'Kiran'}
New on Day 2: {'Meena'}
Did not return on Day 2: {'Asha'}
```

Part 3: Dictionary Programming Questions

Dictionaries store key-value pairs and preserve insertion order in modern Python.

56. Create an employee dictionary and display each value using its key.

SAMPLE INPUT
```
employee = {'id': 101, 'name': 'Asha', 'department': 'IT', 'salary': 45000}
```

EXPECTED OUTPUT
```
ID: 101
Name: Asha
Department: IT
Salary: 45000
```

57. Add a new key-value pair and update an existing value.

SAMPLE INPUT
```
student = {'name': 'Anu', 'marks': 80}
Add age = 21 and change marks to 90.
```

EXPECTED OUTPUT
```
{'name': 'Anu', 'marks': 90, 'age': 21}
```

58. Traverse a dictionary and print all keys and values separately.

SAMPLE INPUT
```
student = {'name': 'Anu', 'age': 21, 'course': 'Python'}
```

EXPECTED OUTPUT
```
Keys: name, age, course
Values: Anu, 21, Python
```

59. Count the number of key-value pairs without using len().

SAMPLE INPUT
```
data = {'a': 10, 'b': 20, 'c': 30, 'd': 40}
```

EXPECTED OUTPUT
```
Number of pairs: 4
```

60. Search for a key in a dictionary and display its value if found.

SAMPLE INPUT
```
employee = {'id': 101, 'name': 'Rahul', 'salary': 50000}
search_key = 'salary'
```

EXPECTED OUTPUT
```
Value: 50000
```

61. Merge two dictionaries without using update(). If keys overlap, values from the second dictionary should replace the first.

SAMPLE INPUT
```
A = {'a': 1, 'b': 2}
B = {'b': 20, 'c': 3}
```

EXPECTED OUTPUT
```
{'a': 1, 'b': 20, 'c': 3}
```

62. Remove a specified key using pop() and display the removed value.

SAMPLE INPUT
```
student = {'name': 'Anu', 'age': 21, 'marks': 85}
key = 'age'
```

EXPECTED OUTPUT
```
Removed value: 21
Remaining: {'name': 'Anu', 'marks': 85}
```

63. Find the key with the highest numeric value without using max().

SAMPLE INPUT
```
scores = {'Asha': 85, 'Rahul': 92, 'Meena': 88}
```

EXPECTED OUTPUT
```
Highest scorer: Rahul
Highest value: 92
```

64. Find the sum and average of all numeric values in a dictionary.

SAMPLE INPUT
```
marks = {'Maths': 80, 'Science': 90, 'English': 70}
```

EXPECTED OUTPUT
```
Sum: 240
Average: 80.0
```

65. Create a dictionary from two lists without using zip().

SAMPLE INPUT
```
keys = ['name', 'age', 'course']
values = ['Asha', 21, 'Python']
```

EXPECTED OUTPUT
```
{'name': 'Asha', 'age': 21, 'course': 'Python'}
```

66. Count the frequency of each character in a string using a dictionary.

SAMPLE INPUT
```
text = 'banana'
```

EXPECTED OUTPUT
```
{'b': 1, 'a': 3, 'n': 2}
```

67. Count the frequency of each word in a sentence.

SAMPLE INPUT
```
sentence = 'python is easy and python is powerful'
```

EXPECTED OUTPUT
```
{'python': 2, 'is': 2, 'easy': 1, 'and': 1, 'powerful': 1}
```

68. Find the character that occurs most frequently in a string. If tied, display all tied characters.

SAMPLE INPUT
```
text = 'success'
```

EXPECTED OUTPUT
```
Most frequent character(s): s
Frequency: 3
```

69. Find all keys that have the same value in a dictionary.

SAMPLE INPUT
```
data = {'Asha': 80, 'Rahul': 90, 'Meena': 80, 'Kiran': 70, 'Ravi': 90}
```

EXPECTED OUTPUT
```
Value 80: ['Asha', 'Meena']
Value 90: ['Rahul', 'Ravi']
```

70. Swap the keys and values of a dictionary. Assume all values are unique.

SAMPLE INPUT
```
data = {'a': 1, 'b': 2, 'c': 3}
```

EXPECTED OUTPUT
```
{1: 'a', 2: 'b', 3: 'c'}
```

71. Create a dictionary containing numbers from 1 to n as keys and their squares as values.

SAMPLE INPUT
```
n = 5
```

EXPECTED OUTPUT
```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

72. Given student marks, display students who scored above the class average.

SAMPLE INPUT
```
marks = {'Asha': 80, 'Rahul': 60, 'Meena': 90, 'Kiran': 70}
```

EXPECTED OUTPUT
```
Average: 75.0
Above average: Asha (80), Meena (90)
```

73. Find the second-highest unique salary in an employee dictionary.

SAMPLE INPUT
```
salaries = {'Asha': 50000, 'Rahul': 65000, 'Meena': 65000, 'Kiran': 58000}
```

EXPECTED OUTPUT
```
Second-highest unique salary: 58000
```

74. Merge two dictionaries and add values when the same key appears in both.

SAMPLE INPUT
```
A = {'apple': 10, 'banana': 5}
B = {'banana': 7, 'orange': 4}
```

EXPECTED OUTPUT
```
{'apple': 10, 'banana': 12, 'orange': 4}
```

75. Sort a dictionary by its values without using sorted().

SAMPLE INPUT
```
data = {'Asha': 75, 'Rahul': 60, 'Meena': 95}
```

EXPECTED OUTPUT
```
Ascending by value: [('Rahul', 60), ('Asha', 75), ('Meena', 95)]
```

76. Create a nested dictionary for five students and calculate each student's total and average across three subjects.

SAMPLE INPUT
```
students = {'Asha': {'Maths': 80, 'Science': 90, 'English': 70}, 'Rahul': {'Maths': 60, 'Science': 75,
'English': 65}}
```

EXPECTED OUTPUT
```
Asha - Total: 240, Average: 80.0
Rahul - Total: 200, Average: 66.67
```

77. Given a nested employee dictionary, find employees earning more than 50000.

SAMPLE INPUT
```
employees = {'E1': {'name': 'Asha', 'salary': 48000}, 'E2': {'name': 'Rahul', 'salary': 62000}, 'E3': {'name':
'Meena', 'salary': 55000}}
```

EXPECTED OUTPUT
```
Rahul: 62000
Meena: 55000
```

78. Create an inventory dictionary. Add products, update stock, remove a product, and display products with fewer than 10 items.

SAMPLE INPUT
```
Start: {'Pen': 20, 'Book': 8}
Add 'Bag': 5; update Pen to 15; remove Book.
```

EXPECTED OUTPUT
```
Low-stock products: Bag (5)
Final inventory: {'Pen': 15, 'Bag': 5}
```

79. Create a shopping cart that tracks quantities and calculates the final bill.

SAMPLE INPUT
```
cart = {'Pen': {'price': 10, 'quantity': 3}, 'Book': {'price': 80, 'quantity': 2}, 'Bag': {'price': 500, 'quantity': 1}}
```

EXPECTED OUTPUT
```
Pen: 30
Book: 160
Bag: 500
Total bill: 690
```

80. Represent departments and employees in a dictionary. Find the department with the largest number of employees.

SAMPLE INPUT
```
departments = {'IT': ['Asha', 'Rahul', 'Kiran'], 'HR': ['Meena', 'Ravi'], 'Sales': ['Anu', 'John', 'Sara', 'Dev']}
```

EXPECTED OUTPUT
```
Department with most employees: Sales
Employee count: 4
```

81. Group a list of words by their first character using a dictionary.

SAMPLE INPUT
```
words = ['apple', 'ant', 'bat', 'ball', 'cat', 'car']
```

EXPECTED OUTPUT
```
{'a': ['apple', 'ant'], 'b': ['bat', 'ball'], 'c': ['cat', 'car']}
```

82. Given student records, create a dictionary grouping students by course.

SAMPLE INPUT
```
records = [('Asha', 'Python'), ('Rahul', 'Java'), ('Meena', 'Python'), ('Kiran', 'Java'), ('Ravi', 'React')]
```

EXPECTED OUTPUT
```
{'Python': ['Asha', 'Meena'], 'Java': ['Rahul', 'Kiran'], 'React': ['Ravi']}
```

83. Create a phone book supporting add, search, update, and delete operations.

SAMPLE INPUT
```
Start: {'Asha': '9876500001', 'Rahul': '9876500002'}
Add Meena: '9876500003'; update Rahul to '9999900002'; delete Asha; search Meena.
```

EXPECTED OUTPUT
```
Meena's number: 9876500003
Final phone book: {'Rahul': '9999900002', 'Meena': '9876500003'}
```

84. Given monthly sales for employees, calculate total yearly sales for each employee.

SAMPLE INPUT
```
sales = {'Asha': {'Jan': 10000, 'Feb': 12000, 'Mar': 8000}, 'Rahul': {'Jan': 15000, 'Feb': 9000, 'Mar': 11000}}
(Only these three months are provided.)
```

EXPECTED OUTPUT
```
Asha total: 30000
Rahul total: 35000
```

85. Create a library dictionary and support borrowing and returning books while tracking available copies.

SAMPLE INPUT
```
books = {'Python Basics': {'author': 'Rao', 'copies': 3}, 'Java Guide': {'author': 'Kumar', 'copies': 1}}
Borrow Python Basics once; borrow Java Guide once; return Java Guide once.
```

EXPECTED OUTPUT
```
Python Basics available copies: 2
```

```
Java Guide available copies: 1
```

Part 4: Mixed Collection

These questions combine tuples, sets, dictionaries, loops, conditions, and built-in functions such as enumerate() and zip().

86. Store student marks in a dictionary and use a set to identify students who passed (marks >= 35).

SAMPLE INPUT
```
marks = {'Asha': 85, 'Rahul': 32, 'Meena': 92, 'Kiran': 30}
```

EXPECTED OUTPUT
```
Passed students: {'Asha', 'Meena'}
Failed students: {'Rahul', 'Kiran'}
```

87. Given a tuple of product records (product_id, name, price), create a dictionary for lookup by product ID.

SAMPLE INPUT
```
products = ((101, 'Pen', 10), (102, 'Book', 80), (103, 'Bag', 500))
Search ID = 102
```

EXPECTED OUTPUT
```
Product found: Book
Price: 80
```

88. Use zip() to create a dictionary from student names and marks, then find the highest scorer.

SAMPLE INPUT
```
names = ['Asha', 'Rahul', 'Meena']
marks = [85, 92, 78]
```

EXPECTED OUTPUT
```
Dictionary: {'Asha': 85, 'Rahul': 92, 'Meena': 78}
Topper: Rahul (92)
```

89. Use a set to find unique words and a dictionary to count the frequency of each word.

SAMPLE INPUT
```
sentence = 'red blue red green blue red'
```

EXPECTED OUTPUT
```
Unique words: {'red', 'blue', 'green'}
Frequency: {'red': 3, 'blue': 2, 'green': 1}
```

90. Given employee records as tuples, identify duplicate employee IDs using a set and create a dictionary of valid records. Keep the first record for each ID.

SAMPLE INPUT
```
employees = ((101, 'Asha'), (102, 'Rahul'), (101, 'Asha R'), (103, 'Meena'))
```

EXPECTED OUTPUT
```
Duplicate IDs: {101}
Valid records: {101: 'Asha', 102: 'Rahul', 103: 'Meena'}
```

91. Given two dictionaries of marks, find students who appeared in both exams and calculate combined totals.

SAMPLE INPUT
```
exam1 = {'Asha': 80, 'Rahul': 70, 'Meena': 90}
exam2 = {'Rahul': 75, 'Meena': 85, 'Kiran': 88}
```

EXPECTED OUTPUT
```
Common students: {'Rahul': 145, 'Meena': 175}
```

92. Given order tuples (customer, amount), use a dictionary to calculate the total amount spent by each customer.

SAMPLE INPUT
```
orders = (('Asha', 250), ('Rahul', 400), ('Asha', 150), ('Meena', 300), ('Rahul', 100))
```

EXPECTED OUTPUT

```
{'Asha': 400, 'Rahul': 500, 'Meena': 300}
```

93. Given products purchased by each customer, use sets to find products purchased by every customer.

SAMPLE INPUT

```
Asha = {'Pen', 'Book', 'Bag'}
Rahul = {'Book', 'Pen', 'Bottle'}
Meena = {'Pen', 'Book', 'Pencil'}
```

EXPECTED OUTPUT

```
Purchased by every customer: {'Pen', 'Book'}
```

94. Given transactions (customer, product, amount), calculate each customer's total spending.

SAMPLE INPUT

```
transactions = (('Asha', 'Pen', 10), ('Rahul', 'Book', 80), ('Asha', 'Bag', 500), ('Rahul', 'Pen', 10))
```

EXPECTED OUTPUT

```
Asha total: 510
Rahul total: 90
```

95. Build a course registration summary using dictionaries for student details and sets for each student's unique course registrations.

SAMPLE INPUT

```
registrations = {'S101': {'name': 'Asha', 'courses': {'Python', 'Java', 'Python'}}, 'S102': {'name': 'Rahul',
'courses': {'Java', 'React'}}, 'S103': {'name': 'Meena', 'courses': {'Python', 'React'}}}
```

EXPECTED OUTPUT

```
S101 Asha: {'Python', 'Java'}
S102 Rahul: {'Java', 'React'}
S103 Meena: {'Python', 'React'}
Students in Python: {'Asha', 'Meena'}
```